Louay MASBAH
Mai 2026

# Rapport 2

# 1. Nouvelles fonctionnalités

## ObstacleSegment (nouvelle classe abstraite)

Création d'une classe abstraite ObstacleSegment étendant Obstacle, destinée aux obstacles définis par deux points A et B (teleporteur et propulseur). Elle fournit les méthodes utilitaires getDirecteur(), getLongueur(), getPointA() et getPointB(), partagées par toutes les sous-classes.

## ObstaclePropulseur — refactoring

Mise à jour pour étendre ObstacleSegment. Le constructeur accepte désormais deux points (pointA, pointB), ce qui permet de dériver la direction d'éjection du vecteur A→B. Cela corrige également le bug qui imposait une direction fixe à 45°.

## ObstacleTeleporteur — nouveau obstacle

Ajout d'un obstacle téléporteur fonctionnel. Deux instances se lient via lier(), et le projectile entrant dans l'une est instantanément déplacé vers l'autre. La zone de détection est circulaire, centrée sur pointA, avec un rayon égal à la distance A→B. Un décalage post-téléportation évite la re-téléportation immédiate.

# 2. Corrections de bugs

Analyse complète du code source existant : 12 problèmes identifiés et corrigés.

| Fichier | Problème | Correction |
|---|---|---|
| Map.java | 10e-9 au lieu de 1e-9 → dt 10× trop grand | Remplacé par 1e-9 (conversion ns→s correcte) |
| Map.java | Boucle bloquante sur l'EDT → UI gelée | Remplacé par un javax.swing.Timer non-bloquant |
| Map.java | estGagne non réinitialisé entre les tours | Ajout de this.estGagne = false dans initialiserParametres() |

| | | |
|---|---|---|
| Map.java | 1000000000L / 60 = division entière tronquée | Remplacé par 1_000_000_000.0 / 60 |
| | | |
| | | |
| CollisionCirculaire.java | System.out.println dans estEnCollision() | Supprimé (60 appels/seconde) |
| CollisionCirculaire.java | centre stocké par référence sans copie | Remplacé par new Vecteur(centre) |
| FenetreJeu.java | Constructeur vide utilisé par ModeEditeur | Identifié comme bug critique (fenêtre vide) |
| | | |

et d'autres bugs que j'ai oublié

# 3. Évolution de l'architecture

Hiérarchie des obstacles clarifiée selon la demande de l'équipe :

- Obstacles à deux points (Propulseur, Téléporteur) : étendent ObstacleSegment qui stocke pointA et pointB.

# 4. Tests unitaires

Fichier ObstacleTeleporteurTest.java ajouté — 14 tests couvrant :

- Liaison réciproque entre deux portails
- Détection de collision circulaire (dedans / dehors)
- Téléportation A→B et B→A
- Absence de re-téléportation immédiate après la sortie
- Comportement sans partenaire (aucun déplacement)
- Rayon contrôlé par pointB
- Méthode copier() : copie non liée, position et rayon conservés

# 5. Utilisation d'AI

J'ai utilisé Claude Code, qui m'a aidé à trouver les bugs et à les comprendre. Ensuite, je l'ai utilisé pour écrire le code des solutions que je proposais. Quelquefois, l'IA proposait des solutions qui pouvaient bouleverser toute l'implémentation des autres parties du code. Sans lecture du code fourni, ou au moins sans compréhension de ce que le code faisait, je

pouvais compliquer le travail des autres. J'ai donc essayé de ne pas laisser l'IA tout régler et de ne pas intégrer une solution sans la comprendre, surtout que j'avais besoin d'expliquer mes modifications aux autres membres dont le travail serait affecté par celles-ci.

Néanmoins, l'utilisation de l'IA m'a permis d'accomplir beaucoup de travail sans avoir à faire un problem solving vraiment complexe, sauf dans quelques rares cas.